Catching an FP16 Training Divergence with nixnan

From the first large value to the first NaN in TinyGPT

SC26 nixnan tutorial

`flash-attn-expt/fp16-divergence`, branch `Flash-Attn`

October 2026

The task

- Train TinyGPT **entirely in FP16**: parameters, activations, gradients
  - no FP32 master weights, no loss scaling (`GradScaler`), no clipping
- Use a learning rate **high enough to diverge**
- With **nixnan** (binary instrumentation, no source changes), catch the sequence of values that leads to ∞ and NaN
- **Stop as soon as the NaN is seen**

FP16 in one line

max finite $= 65{,}504$ (top binade 2^{15}) min normal $= 2^{-14}$ $|v| \geq 65{,}520 \Rightarrow \pm\infty$

Answer: **yes**. nixnan shows the climb three steps early, the first ∞ and the first NaN, with the SASS instruction for each.

Setup

TinyGPT

- 4 pre-LN blocks, $d = 128$, 4 heads, MLP 512, RoPE, no biases
- 791,808 parameters, 12-token vocabulary (digits)
- 100 sequences $\times$ 21 positions, one batch
- attention: SDPA, flash backend
- two `.float()` casts now follow the weight dtype (FP32 model unchanged: 312 $\rightarrow$ 312452)

`train_fp16.py`

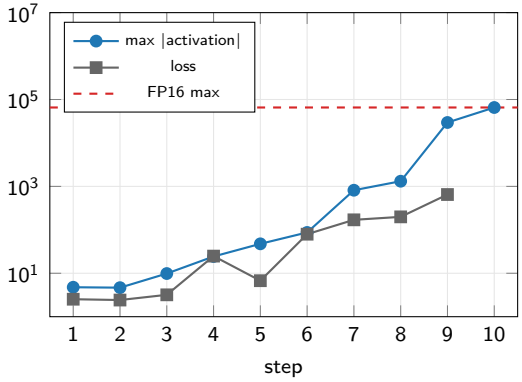
```
model = TinyGPT().cuda().half()   # no master weights
opt = torch.optim.SGD(model.parameters(),
                      lr=1.0, momentum=0.9)
for step in range(1, 501):
    logits = model(x)
    loss = F.cross_entropy(logits...float(), y)
    opt.zero_grad(); loss.backward()
    print(f"step {step:4d}  loss {loss.item():.4f}")
    if not finite(loss): break      # STOP
    opt.step()
```

SGD: update $= \eta \cdot$gradient, so weights grow freely.
Deterministic: identical losses on every run.

Ground truth: the oracle (`-oracle`, not under nixnan)

step	loss	max |act|	largest
1	2.53	4.8	`blocks.1.ln2`
4	24.8	24	`head`
6	79.3	87	`head`
7	169.9	815	`blocks.0.proj`
8	198.8	1,315	`blocks.0.proj`
9	649.4	29,570	`blocks.0.proj`
10	NaN	65,440 + 7,870 ∞	`blocks.0.proj`

First non-finite tensor at step 10:
output of `blocks.0.proj`: 7,870 ∞, 0 NaN



The nixnan setup: three runs and a watcher

```
# pass 0: log all 50 kernel names (dummy whitelist: nothing instrumented) -> generate the lists
# pass 1: exceptions everywhere except 18 tile-based kernels
watch_nan.py logs/nixnan-exceptions.log -- env LD_PRELOAD=nixnan.so FUNCTION_BLACKLIST=blacklist.txt
    python train_fp16.py --lr 1
# pass 2: binade histogram + instruction, on the residual add and LayerNorm only
watch_nan.py logs/nixnan-climb.log -- env LD_PRELOAD=nixnan.so FUNCTION_WHITELIST=hist-whitelist.txt
    \
     BIN_SPEC_FILE=spec-climb.json HISTOGRAM=1 python train_fp16.py --lr 1
```

spec-climb.json

```
{ "count": 1, "doublings": 20,
  "f16 (record_inst)": [[9,9],...,[15,15]],
  "f32 (record_inst)": [[9,9],...,[15,15],[16,17]]
    }
```

one bin per binade 2^9–2^{15}; report the first hit and the instruction

watch_nan.py

- timestamps every line of output
- first line matching
  `#nixnan: error \[[^\]]*NaN`
  → `WATCHER` line, `SIGKILL` the process group
- subnormal and $\pm\infty$ reports don't stop it

Result: the timeline nixnan reports (`logs/SUMMARY.txt`)

step	loss	residual add (`CUDAFunctor_add<Half>`)	LayerNorm	exceptions
1–6	2.5–79	below 2^9	up to 2^{17} (FP32 $\sum(x-\mu)^2$)	none
7	169.9	2^9 **($\geq$ 512)**, `FFMA`	2^{17}	none
8	198.8	2^{10} **($\geq$ 1,024)**	2^{17}	none
9	649.4	2^{15} **($\geq$ 32,768)**, 49k reports	2^{17}	none
10	—	2^{15}, then $\pm\infty$ in `FFMA`	$-\infty$, then **NaN**	**run killed**

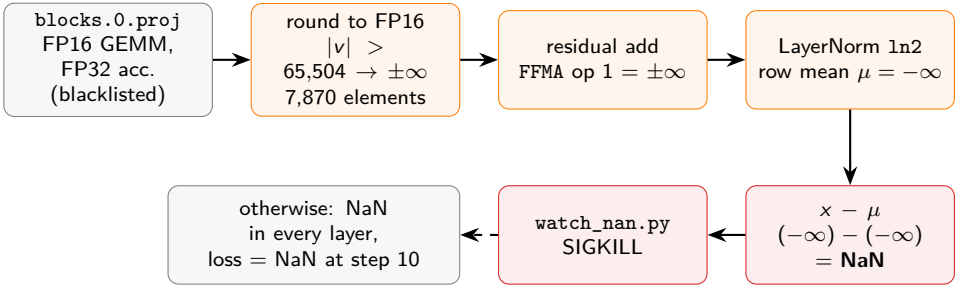
- **Early warning**: the residual stream climbs binade by binade, 3 steps before overflow
- Oracle maxima 815 / 1,315 / 29,570 fall in exactly these binades
- FP16 bins never fire: PyTorch computes in **FP32** after loading FP16

Step 10, report by report

```
[54.0s] error [infinity]  operand 1 of FFMA R9, R9, c[0x0][0x164], R8   ; add kernel
[54.0s] error [infinity]  operand 0 of FFMA R9, R9, c[0x0][0x164], R8   ; (3 FFMAs, +inf and -inf)
[54.0s] error [-infinity] operand 1 of FADD R18, R19, -R16              ; LayerNorm
[54.0s] error [-infinity] operand 0 of FFMA R16, R18, R15, R16
[54.0s] error [-infinity] operand 1 of FADD R0, R19, -R16
[54.0s] error [-infinity] operand 2 of FADD R0, R19, -R16
[54.0s] error [NaN]       operand 0 of FADD R0, R19, -R16
[54.0s] WATCHER: first nixnan NaN report -- killing the run
```

- Operand 0 = destination, 1, 2, ... = sources
- Add: $d = \alpha \cdot b + a$ with $\alpha = \texttt{c[0x0][0x164]}$; $b = \texttt{proj}$ output $= \infty$. **The add reads the ∞; it doesn't create it**
- LayerNorm: running mean $\mu \to -\infty$, then $x - \mu = (-\infty) - (-\infty) =$ **NaN**

How an overflow becomes a NaN



- nixnan sees the GEMM's overflow **one kernel later**, as an ∞ source operand
- The oracle names the producer: `blocks.0.proj` (first non-finite tensor, forward order)
- Exceptions pass (32 kernels, 213 s) agrees; first NaN printed by the FFMA consuming $x - \mu$ (warp order)

Obstacle 1: false NaNs in a healthy step 1

Instrument everything $\Rightarrow$ the watcher fires at **step 1**, while every tensor is finite and ≤ 5.

```
error [NaN,subnormal] operand 1 of HMMA.16816.F32 R20, R48.reuse, R46, R20 ; void cutlass::Kernel2  (
    f16)
```

GEMMs blacklisted $\Rightarrow$ next stop, `softmax_warp_forward` in `cross_entropy`:

```
error [-infinity] operand 0 of FFMA R8, R7, 1.4426950216293334961, -R8 ; void   (log2 e)
error [NaN]       operand 0 of FADD R7, -R12, R7 ; void
```

- **Padding lanes**: tile lanes outside the tensor compute on stale bits (GEMM) or on $-\infty$ padding (softmax: $-\infty - (-\infty)$), and are never stored
- Same effect as flash attention (`PyTorch_Instrumentation/PyT.md`)
- Fix: blacklist 18 tile kernels (cuBLAS, CUTLASS, splitK, flash, warp softmax).
  A GEMM overflow is still caught, by the next kernel

Obstacles 2–4: what the run says about nixnan

2. `MAX_ERRORS` can't stop at a NaN

- counts every report, subnormals included
- FP16 GEMMs report subnormals at step 1; GELU at step 8
- → `watch_nan.py`; suggestion: `MAX_NANS` / category mask

4. Truncated kernel names

- `cut_kernel_name`: `find_first_of("<(")`
- LayerNorm prints as `void at::native::`, softmax as `void`
- identifiable only because pass 2 instruments 2 kernels

3. `HISTOGRAM=1` on all kernels segfaults

```
// src/recording.cu:88  recorder::mk_entry
inst_info[current_entry].instr = instr;
...
return current_entry++;   // no bound check
// src/nixnan.cu:71
#define TABLE_SIZE (1l << 17)
```

- gdb: `SIGSEGV` in `mk_entry`, `FFMA` of `cutlass::Kernel2`
- large GEMMs overflow the 2^{17}-entry table
- → histogram whitelist of 2 kernels; nixnan left unmodified

Reproducibility and caveats

- `./run_nixnan.sh`: ≈5 min on an RTX 3090; full rerun gave
  - the same losses, the same histogram timeline, the same step-10 `FFMA`/`FADD` sequence
  - only difference: *which* LayerNorm instruction prints the first NaN (`FADD R0,R19,-R16` or the `FFMA R17,R18,R0,R17` that consumes it), warp order
- LayerNorm's FP32 bins reach 2^{17} from step 6 on: variance sums, harmless.
  The residual add is the signal to watch
- nixnan cannot see *inside* the blacklisted GEMM; the oracle names it
- Environment: PyTorch 2.6.0+cu124, CUDA 12.4, sm_86, NVBit 1.8, nixnan unmodified

Takeaways

- Without touching the program, nixnan gave:
  - the **climb**: residual stream ≥ 512, $\geq 1{,}024$, $\geq 32{,}768$ in steps 7, 8, 9
  - the **first** ∞: the residual add's `FFMA`, operand 1 (an overflowed GEMM output)
  - the **first NaN**: LayerNorm's $x - \mu = (-\infty) - (-\infty)$, then the run was killed
- Aiming it took a model of the program: which kernels are tile-based, which carry the residual stream, and that FP16 is computed as FP32
- Tool improvements suggested by this run: padding-lane awareness, `MAX_NANS`, a bounded instruction table, full kernel names
- Standard fixes break one link each: lower lr / AdamW, clipping, master weights + `GradScaler`, or BF16

Report: `fp16-divergence/latex/report.pdf` Code and logs: `flash-attn-expt/fp16-divergence/`